

Atlantic Meats Project

A custom-built asset and maintenance management system designed specifically for a Atlantic Meats to streamline the management of assets across multiple store locations. The platform centralises all maintenance activity into a single digital system, replacing manual WhatsApp tracking methods with structured, traceable job cards and a unified asset register.

It provides a consolidated view of all assets across stores, enabling clear visibility into maintenance history, ongoing work, and asset performance. Each maintenance action is recorded digitally, ensuring a complete and reliable history of work performed on every asset.

Aim of the Platform

The primary aim is to reduce maintenance expenditure and improve the efficiency and accuracy of asset management across all customer store locations.

Key objectives include:

- Reducing overall maintenance spend through improved visibility and preventative insights
- Providing store-level and asset-level insights into maintenance activity and recurring issues
- Centralising asset management across all customer locations into a single system
- Replacing manual processes with digital job cards for consistent and traceable maintenance records
- Maintaining a complete maintenance history per asset to support troubleshooting and decision-making
- Identifying patterns and root causes of recurring faults to enable long-term corrective actions

Table of Contents

- [Introduction](#)
- [Tech Stack](#)
- [Features](#)
- [Project Structure](#)
- [Installation](#)
 - [Clone the Repository](#)
 - [Navigate to the project](#)
 - [Install Dependencies](#)
 - [Start Development Server](#)
- [Environment Variables](#)
- [Github Secrets](#)
- [Available Scripts](#)

- [Build](#)
- [Deployment](#)
- [ESLint Configuration](#)
- [Usage](#)
- [API Reference](#)
- [Application Modules](#)
 - [Jobs Module](#)
 - [Overview](#)
 - [Problem](#)
 - [Solution](#)
 - [User Experience](#)
 - [Scope](#)
 - [Architecture](#)
 - [Infrastructure](#)
 - [Prerequisites](#)
 - [Open Questions](#)
 - [Assets Module](#)
 - [Overview](#)
 - [Problem](#)
 - [Solution](#)
 - [User Experience](#)
 - [Scope](#)
 - [Architecture](#)
 - [Infrastructure](#)
 - [Prerequisites](#)
 - [Open Questions](#)
 - [Assets Transfer Module](#)
 - [Overview](#)
 - [Problem](#)
 - [Solution](#)
 - [User Experience](#)
 - [Scope](#)
 - [Architecture](#)
 - [Infrastructure](#)
 - [Prerequisites](#)
 - [Open Questions](#)
 - [Stock Management Module](#)
 - [Overview](#)

- Problem
 - Solution
 - User Experience
 - Scope
 - Architecture
 - Infrastructure
 - Prerequisites
 - Open Questions
 - Comments Module
 - Overview
 - Problem
 - Solution
 - User Experience
 - Scope
 - Architecture
 - Infrastructure
 - Prerequisites
 - Open Questions
 - Users Module
 - Overview
 - Problem
 - Solution
 - User Experience
 - Scope
 - Architecture
 - Infrastructure
 - Prerequisites
 - Open Questions
 - WhatsApp Comment Notifications MOdule
 - Overview
 - Problem
 - Solution
 - User Experience
 - Scope
 - Architecture
 - Infrastructure
 - Prerequisites
 - Open Questions
 - Contact
-

Tech Stack

- React
- TypeScript
- Vite
- Tailwind CSS
- React Router Dom
- React Query
- AWS
- Terraform

Features

- User authentication
- Responsive dashboard
- API integration
- Github actions deployment
- Role-based access

Project Structure

```
src/  
├── assets/  
├── auth/  
├── components/  
├── context/  
├── data/  
├── hooks/  
├── lib/  
├── pages/  
├── routes/  
├── schemas/  
├── styles/  
└── utils/
```

Installation

Clone the repository

```
git clone https://github.com/Fabian-Petersen/atlantic-meats-app-fabian-portfolio.git
```

Navigate to the project

```
cd atlantic-meats-app
```

Install dependencies

```
npm install
```

Start development server

```
npm run dev
```

Application runs on:

```
http://localhost:5173
```

Environment Variables

Create a `.env` file in the root directory:

```
VITE_COGNITO_USERPOOL_ID=  
VITE_COGNITO_CLIENT_ID=  
VITE_SITE_URL=
```

Github Secrets

```
API_GATEWAY_URL= AWS_ACCESS_KEY_ID= AWS_S3_BUCKET= AWS_SECRET_ACCESS_KEY=  
CLOUDFRONT_DISTRIBUTION_ID= COGNITO_CLIENT_ID= COGNITO_USERPOOL_ID=
```

Available Scripts

Command	Description
<code>npm run dev</code>	Start development server

Command	Description
<code>npm run build</code>	Create production build
<code>npm run preview</code>	Preview production build
<code>npm run lint</code>	Run ESLint

Build

```
npm run build
```

Production files are generated in:

```
dist/
```

Deployment

Example deployment platforms:

- AWS Amplify
- AWS S3 + CloudFront

ESLint Configuration

This project uses ESLint with TypeScript support.

Example plugins:

- eslint-plugin-react
- eslint-plugin-react-hooks
- typescript-eslint

Usage

Content here...

API Reference

Content here...

Application Modules

Jobs Module

Overview

Problem

Solution

User Experience

Scope

Architecture

Infrastructure

Prerequisites

Open Questions

Assets Module

Overview

Problem

Solution

User Experience

Scope

Architecture

Infrastructure

Prerequisites

Open Questions

Assets Transfer Module

Overview

The Assets Transfer Module provides a controlled process for transferring assets between business units, facilities, departments, or custodians.

The module ensures that all transfers are formally requested, approved, tracked, and acknowledged before the asset record is updated.

The solution is implemented using an event-driven serverless architecture on AWS and integrates with the Asset Management System.

Problem

Prior to implementing the transfer workflow, asset movements could occur without a formal approval process or reliable audit trail.

This introduced several challenges:

- Assets could be moved without authorization.
 - There was limited visibility into who requested or approved a transfer.
 - Asset locations could become inaccurate.
 - No confirmation existed that the receiving party had physically received the asset.
 - Auditing and compliance reporting required manual investigation. The organization required a process that would provide accountability and traceability throughout the entire transfer lifecycle.
-

Solution

The Assets Transfer Module introduces a controlled workflow consisting of:

1. Transfer Request Submission
2. Administrative Approval or Rejection
3. Transport Arrangement
4. Transfer Initiation
5. Recipient Acknowledgement
6. Asset Location Update

Every transfer is recorded in the `asset_transfer_table`, creating a complete history of:

- Requestor
- Recipient
- Approver
- Transfer reason
- Approval decision
- Transfer dates
- Current transfer status

Asset records are not updated when approval occurs.

Approval only authorizes the movement of the asset.

The asset remains assigned to the source location until the requestor confirms the asset has physically been dispatched.

When dispatch occurs, the transfer status changes to:

APPROVED → IN_TRANSIT

Only after the recipient confirms receipt does the workflow progress to:

IN_TRANSIT → RECEIVED

At this point the asset record is updated to reflect the new ownership or location.

All state transitions are enforced using DynamoDB conditional updates to prevent invalid transitions and duplicate processing.

Valid status transitions are:

```
PENDING → APPROVED
PENDING → REJECTED
PENDING → EXPIRED
APPROVED → CANCEL
APPROVED → IN_TRANSIT
IN_TRANSIT → RECEIVED
```

User Experience

Requestor

The requestor can:

- Initiate a transfer request
- Specify recipient and destination
- Provide a transfer reason
- Initiate the transfer once transport has been arranged
- Track transfer status Typical workflow:

```
Create Request
  ↓
Await Approval
  ↓
Receive Approval/Rejection Notification
  ↓
Arrange Transport → Mark In Transit
  ↓
Await Recipient Confirmation
```

Administrator

The administrator can:

- Review pending requests
 - Approve requests
 - Reject requests
 - Cancel an approved transfer before it is in transit
 - Receive reminder notifications for pending approvals
- Typical workflow:

```
Receive Notification
↓
Review Request
↓
Approve, Reject, or Cancel
```

Recipient

The recipient can:

- Receive transfer notifications
 - Confirm asset receipt
 - Complete the transfer process
- Typical workflow:

```
Receive Notification
↓
Physically Receive Asset
↓
Confirm Receipt
```

Scope

Included

- Asset transfer requests
- Approval workflow
- Approval cancellation (post-approval, pre-transit)
- Approval reminders
- Transfer rejection workflow
- Recipient acknowledgement
- Asset location updates
- Transfer audit history
- In-app notification delivery (via `notifications_table` / `GET /notifications`)
- Email notification delivery (via SNS)

Excluded

- Physical asset transportation
- Asset maintenance activities
- Asset disposal workflows
- Asset procurement workflows
- Multi-stage approval chains

Architecture

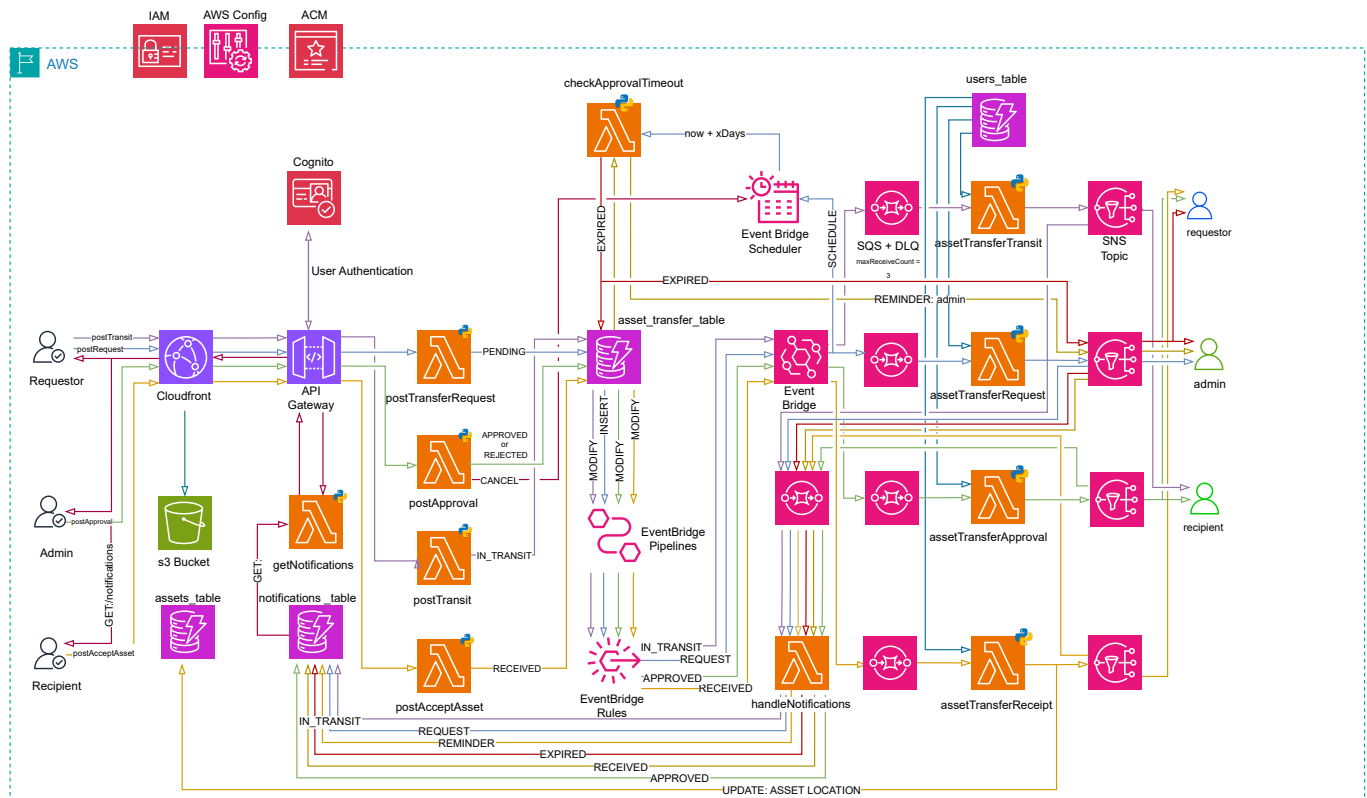


Figure 1: Asset Transfer Module Architecture

High-Level Workflow

```

Requestor
↓
Transfer Request
↓
Admin Approval
↓
Requestor Marks In Transit
↓
Recipient Notification
↓
Physical Transfer
↓
Recipient Confirmation
  
```

↓
Asset Location Update

Transfer Status Lifecycle

PENDING
↓
APPROVED
↓
IN_TRANSIT
↓
RECEIVED

Alternative outcomes:

PENDING
↓
REJECTED

PENDING
↓
EXPIRED

APPROVED
↓
CANCEL

Status transitions are protected using DynamoDB conditional writes to ensure that:

- Duplicate events cannot update a transfer more than once.
- Invalid workflow transitions are rejected.
- Lambda retries do not result in inconsistent data. Examples:

PENDING → APPROVED ✓
PENDING → REJECTED ✓
PENDING → EXPIRED ✓

APPROVED → IN_TRANSIT ✓
APPROVED → CANCEL ✓

IN_TRANSIT → RECEIVED ✓

```
PENDING → RECEIVED X
PENDING → IN_TRANSIT X
REJECTED → APPROVED X
EXPIRED → APPROVED X
RECEIVED → IN_TRANSIT X
```

Key Design Principles

Controlled Asset Movement

Asset locations are not updated when a request is approved.

Approval indicates authorization to move the asset but does not confirm that the asset has physically left the source location.

Once transportation has been arranged and the asset is physically dispatched, the requestor initiates the transfer and the status is updated to `IN_TRANSIT`.

The asset location is only updated after the recipient acknowledges receipt and the transfer status changes to `RECEIVED`.

Auditability

All transfer actions are recorded in the `asset_transfer_table`, including:

- Who requested
- Who approved
- Who received
- Why the transfer occurred
- When actions occurred

Event-Driven Processing

Writes to the API land directly in `asset_transfer_table`. DynamoDB Streams capture every `INSERT/MODIFY` and feed an **EventBridge Pipe**, which forwards each change as an event onto a custom **EventBridge** bus.

EventBridge Rules pattern-match on the new transfer status (`PENDING/REQUEST`, `APPROVED`, `IN_TRANSIT`, `RECEIVED`, `EXPIRED`, `REMINDER`) and fan out to the appropriate downstream Lambda(s), decoupling the write path from notification and side-effect processing.

Notification Delivery (Two Channels)

Two notification surfaces are maintained in parallel:

- **In-app notifications** — `handleNotifications` consumes status-change events and writes records to `notifications_table`, surfaced to users via `GET /notifications` (`getNotifications` Lambda).
- **Email notifications** — four status-specific Lambdas (`assetTransferRequest`, `assetTransferApproval`, `assetTransferTransit`, `assetTransferReceipt`) each publish to their own dedicated SNS topic, which delivers to the relevant actor(s) (requestor, admin, recipient). `assetTransferTransit` is additionally buffered behind an SQS queue with a DLQ.

`assetTransferReceipt` carries the extra responsibility of updating the asset's location in `assets_table` once receipt is confirmed.

Approval Timeouts

When a request is approved, an **EventBridge Scheduler** one-time schedule is created for `now + xDays`. If the request has not progressed to `IN_TRANSIT` by then, `checkApprovalTimeout` fires, writes `EXPIRED` to `asset_transfer_table`, and emits a reminder event back onto the EventBridge bus for the admin notification path.

Fault Tolerance

An SQS queue with a dead-letter queue (`maxReceiveCount = 3`) sits in front of the `assetTransferTransit` Lambda, providing retry capability and preventing message loss on transient failures.

Data Model

Single-Item, Progressive Enrichment Pattern

Each transfer is represented by **one item** in `asset_transfer_table`, keyed by `transferId`. The item is created at request time with a `PENDING` status and a base set of fields, then progressively enriched in place as the transfer advances — `approval`, `inTransit`, and `receipt` are nested attribute blocks added to the same item at each respective stage, rather than separate records. This keeps the full transfer history and current state co-located on one item, which is what makes the item itself double as the audit trail described under Auditability above.

Fields fall into two categories:

- **Client-supplied** — provided by the requestor/recipient/admin through the API (e.g. `recipientSub`, `transferReason`, `condition`).
- **Backend-derived** — set by the Lambda from the authenticated Cognito session or server clock, never trusted from client input (e.g. `requestorSub`, `dateApproved`, `approvedBySub`). This distinction matters for both validation (reject any client attempt to set a backend-derived field) and for IAM/Cognito claim mapping in the Lambda implementation.

`asset_transfer_table`

```
{
  "transferId": "string (PK)",
  "transferCreated": "string (ISO 8601, backend-derived)",
  "status": "PENDING | APPROVED | REJECTED | EXPIRED | CANCEL | IN_TRANSIT | RECEIVED",

  "requestorSub": "string (backend-derived from Cognito claim)",
  "approverSub": "string | null (intended/assigned approver, set at request time)",
  "recipientSub": "string (client-supplied)",

  "description": "string",
```

```

"assetId": "string",
"transferReason": "string",
"locationFrom": "string",
"locationTo": "string",
"expectedDate": "string (ISO 8601, client-supplied)",

"approval": {
  "dateApproved": "string (ISO 8601, backend-derived)",
  "approvedBySub": "string (backend-derived from Cognito claim)",
  "status": "APPROVED | REJECTED"
},

"inTransit": {
  "dateCreated": "string (ISO 8601, backend-derived)",
  "inTransitSub": "string (backend-derived from Cognito claim)",
  "status": "IN_TRANSIT",
  "transferDate": "string (ISO 8601, client-supplied)",
  "transportType": "company | courier | contractor | individual",
  "transportName": "string",
  "transportCost": "number | null",
  "invoiceUrl": "string | null (optional attachment)",
  "imageUrls": "string[] | null (optional)"
},

"receipt": {
  "dateReceived": "string (ISO 8601, backend-derived)",
  "receivedBySub": "string (backend-derived from Cognito claim)",
  "status": "RECEIVED",
  "condition": "excellent | damaged",
  "damageDetails": "string | null (required if condition = damaged)",
  "imageUrls": "string[] | null (optional)",
  "deliveryNoteUrl": "string | null (optional)"
}
}

```

notifications_table

```

{
  "notificationId": "string (PK)",
  "status": "PENDING | APPROVED | REJECTED | EXPIRED | CANCEL | IN_TRANSIT | RECEIVED",
  "sub": "string (Cognito sub of the recipient of this notification)",
  "notificationDate": "string (ISO 8601, backend-derived)"
}

```

Open Modeling Questions

Frontend

Service	Purpose
CloudFront	Content delivery and secure access
S3	Static website hosting

Authentication

Service	Purpose
Cognito	Authentication and authorization

API Layer

Service	Purpose
API Gateway	Secure API endpoints

Compute

Lambda Function	Purpose
postTransferRequest	Create transfer request (status: PENDING)
postApproval	Process approval decision (APPROVED / REJECTED / CANCEL)
postTransit	Mark transfer in transit (IN_TRANSIT)
postAcceptAsset	Process recipient acknowledgement (RECEIVED)
getNotifications	Retrieve in-app notifications for the current user
handleNotifications	Write in-app notification records to <code>notifications_table</code>
assetTransferRequest	Notify admin of a new transfer request via SNS
assetTransferApproval	Notify requestor and recipient of approval/rejection via SNS
assetTransferTransit	Notify requestor that the transfer is in transit via SNS (via SQS)
assetTransferReceipt	Notify recipient/requestor of receipt and update <code>assets_table</code>
checkApprovalTimeout	Detect approval expiry and emit EXPIRED / reminder events

Data Layer

Table	Purpose
asset_transfer_table	Transfer workflow and audit history
assets_table	Current asset information

Table	Purpose
notifications_table	In-app notification records
users_table	Provide details of users to be notified

Event Processing

Service	Purpose
DynamoDB Streams	Captures changes on <code>asset_transfer_table</code>
EventBridge Pipes	Routes stream records onto the custom event bus
EventBridge	Custom event bus and rule-based routing
EventBridge Rules	Status-based fan-out to handler Lambdas
EventBridge Scheduler	Approval timeout scheduling
SQS	Event buffering and retries (transit path)
DLQ	Failed message handling (maxReceiveCount = 3)
SNS	Email notification delivery (4 status topics)

Prerequisites

The following components must exist before the module can operate:

- Asset Management Module
- Asset Registry (`assets_table`)
- User Authentication (Cognito)
- User Roles and Permissions
- Email Notification Configuration
- EventBridge Infrastructure
- SQS Queues and DLQs
- SNS Topics and Subscriptions Required roles:
 - Requestor (User, Manager or Admin)
 - Administrator (Admin)
 - Recipient (User, Manager)

Open Questions

Business Questions

- What is the approval expiry period?
- How many reminder notifications should be sent?
- Can administrators override expired requests?
- Should transfers support multiple recipients?
- What is the cancellation policy once a transfer is APPROVED but not yet IN_TRANSIT?

Technical Questions

- Should transfer history be immutable?
- Should approval comments be mandatory?
-
- Should transfer events be exposed for reporting integrations?
- Should support be added for multi-level approvals in future?
- Should the `assetTransferRequest/Approval/Transit` notification Lambdas also sit behind SQS + DLQ, consistent with `assetTransferTransit`?

Future Enhancements

- Multi-stage approval workflows
- Escalation paths for overdue approvals
- Transfer analytics dashboard (no transfers completed, most transfers by location, no pending transfers)
- Mobile approval workflow
- QR code-based transfer confirmation (scan asset to confirm location and barcode number)
- Integration with maintenance workflows

Architecture Review Notes & Recommendations

The points below were identified and evaluated against AWS best practices for event-driven serverless systems (reliability, security, operational excellence, and cost pillars of the Well-Architected Framework).

Recommendation:

Have notification Lambdas check/record an idempotency key (e.g. `transferId#status` written to a small DynamoDB idempotency table with a TTL, or conditional-put against `notifications_table`) before publishing to SNS, to avoid double-notifying a recipient on retried events. This is cheap to add now versus a confusing "why did I get two emails" bug report later.

1. Approval cancellation path needs an explicit notification rule

The status diagram now includes `APPROVED` → `CANCEL`, but it's not clear from the diagram which EventBridge Rule/Lambda picks up `CANCEL` for notification purposes — it doesn't appear to map cleanly to `assetTransferApproval` (which seems oriented around `APPROVED/REJECTED`) or any other existing topic.

Recommendation:

Decide explicitly whether `CANCEL` reuses the `assetTransferApproval` topic (with a distinguishing message attribute) or needs its own rule/topic, and document it. This is one of the nine discrepancies worth nailing down before Terraform, since IAM policies and SNS filter policies need to know about it upfront.

2. Observability: no mention of structured logging, tracing, or alarms

The architecture has six EventBridge Rules, an EventBridge Pipe, a Scheduler, SQS+DLQ, and nine Lambdas. With this much asynchronous fan-out, **AWS X-Ray tracing** (or at minimum correlation IDs propagated through `detail` payloads) becomes important for debugging "why didn't the recipient get notified" support questions.

Recommendation:

Add a `transferId`-keyed correlation ID to every event `detail` payload from the source Lambda all the way through to SNS message attributes, and enable X-Ray active tracing on the API Gateway → Lambda → EventBridge chain. Also add a CloudWatch Alarm on the DLQ's `ApproximateNumberOfMessagesVisible` (you already have the DLQ — alarming on it is the missing half) so failed transit notifications surface immediately rather than being discovered during an audit.

3. Least-privilege IAM scoping given prior `EntityAlreadyExists` history

Given the noted history of `EntityAlreadyExists` IAM issues, when writing the Terraform for these ~9 Lambdas it's worth scoping each Lambda's execution role to only the specific table/topic/queue ARNs it touches (rather than a shared broad role), both for security and to reduce the chance of cross-module naming collisions during `terraform plan/apply`. A `for_each`-driven module per Lambda with explicit `dynamodb:Query/PutItem/UpdateItem` actions scoped to the specific table ARN (and index ARN where GSIs are used) will also make future audits straightforward given how central auditability is to this module's purpose.

4. Confirm EventBridge Scheduler cleanup

One-time schedules created per transfer request for `checkApprovalTimeout` should be deleted once a transfer leaves `PENDING` (either by being approved, rejected, or expiring) — otherwise stale schedules accumulate. Worth confirming the approval/rejection Lambda also deletes the corresponding Scheduler schedule, not just the expiry Lambda consuming it.

Stock Management Module

Overview

Problem

Solution

User Experience

Scope

Architecture

Infrastructure

Prerequisites

Open Questions

Comments Module

Overview

Problem

Solution

User Experience

Scope

Architecture

Infrastructure

Prerequisites

Open Questions

Users Module

Overview

Problem

Solution

User Experience

Scope

Architecture

Infrastructure

Prerequisites

Open Questions

WhatsApp Comment Notifications Module

Overview

When a comment is added to a job, all relevant users receive a WhatsApp message containing the comment — no login required.

Problem

Users miss job updates because they need to log in to the app to see new comments. This creates delays in communication and response times.

Solution

Automatically send a WhatsApp notification to all users when a comment is posted on a job. The message mirrors what is shown in the app, giving users the full context directly in WhatsApp.

User Experience

Trigger: A comment is posted on a job via `POST /comments`.

What the user receives:

New comment on Job-BTX-20260501-0001

"Please confirm the site visit for Thursday morning.
Materials have been delivered."

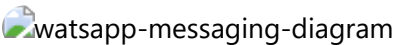
Leon Mathlay (Maintenance Manager)
Posted: 13 May 2025, 09:42

Users see the job reference, full comment, author, and timestamp — no need to open the app.

Scope

In scope	Out of scope
Notify all users of the store plus admin	Per-user notification preferences
Send on every new comment	Edit / delete comment notifications
Include job ID, comment, author, timestamp	Two-way WhatsApp replies

Architecture



Key design decisions:

- **Async delivery** — the API returns `201` immediately; WhatsApp sending happens out of band via Lambda.
- **AWS End User Messaging Social** — native AWS service, no third-party WhatsApp providers needed.
- **Fan-out via Lambda** — one Lambda invocation fetches all users by location (including admin users) and sends messages in parallel.

Infrastructure

Component	Service
API	AWS APIGateway
Database	AWS DynamoDB
Event bus	AWS EventBridge

Component	Service
Notification function	AWS Lambda (Python 3.12)
WhatsApp delivery	AWS End User Messaging Social
Infrastructure-as-code	Terraform

Prerequisites

- WhatsApp Business Account (WABA) linked in AWS End User Messaging Social
- Users must have a `phoneNumber` attribute in the Users table
- WhatsApp message template approved by Meta (required for outbound notifications)

Open Questions

- Should certain roles be excluded from notifications (e.g. admins who posted the comment)?
- Do we notify only users associated with the job, or all users?
- What happens if a user's phone number is invalid or unregistered on WhatsApp?

Contact

Fabian Petersen

Last updated: May 2025